DAY 06 | POST-READ

FastAPI Core and ORM: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Path params	Part of the URL, required, identify a resource: /books/{book_id}.
Query params	After ?, often optional, used to filter and paginate. Query() adds rules.
Body	JSON sent with POST, PUT, PATCH. Declared as a Pydantic model.
APIRouter	Group routes by feature with prefix and tags, then include_router.
Pydantic	BaseModel with typed fields; Field() for rules; invalid data returns 422.
Model variants	Create (no id), Update (all optional), Response (safe fields only).
response_model	Filters and documents what an endpoint returns.
PATCH pattern	model_dump(exclude_unset=True) then model_copy(update=...).
ORM	Classes map to tables, objects to rows. Engine, Base, models, Session, commit.

Part B: Going beyond today

1. Custom validators

```
from pydantic import field_validator

class BookCreate(BaseModel):
    isbn: str

    @field_validator("isbn")
    @classmethod
    def check_isbn(cls, v):
        if len(v.replace("-", "")) != 13:
            raise ValueError("ISBN must have 13 digits")
        return v
```

2. Enums for fixed choices

```
from enum import Enum
class Genre(str, Enum):
    fiction = "fiction"
    science = "science"

class BookCreate(BaseModel):
    genre: Genre
```

Only listed values are accepted, and /docs shows a dropdown.

3. Querying with SQLAlchemy 2.0 style

```
from sqlalchemy import select
with Session(engine) as s:
    stmt = select(BookRow).where(BookRow.pages > 200).order_by(BookRow.title)
    for book in s.scalars(stmt):
        print(book.title)
```

4. from_attributes: ORM object to Pydantic

```
class Book(BaseModel):
    model_config = {"from_attributes": True}
    id: int
    title: str

Book.model_validate(book_row)   # works with SQLAlchemy objects
```

This is how you return database rows through FastAPI response models. You will use it tomorrow.

5. Why ORMs prevent SQL injection

Building SQL by joining strings with user input lets attackers inject commands. ORMs send values separately from the SQL text (parameterised queries), which closes this hole.

Part C: Check yourself

- How does FastAPI decide whether a parameter is a path, query or body parameter?
- Why do we use separate Create and Response models?
- What does exclude_unset=True do?
- What is the job of a Session in SQLAlchemy?
- Name two benefits of using an ORM.

Part D: Practice ideas for tonight

- Add a members router with its own Pydantic models to your Books API.
- Add a genre Enum and a validator that rejects titles in all capitals.
- Store books in SQLite with SQLAlchemy and list books with more than 200 pages.

Coming up on Day 7: SQLAlchemy models and relationships, transactions, and polishing your API documentation with Swagger and OpenAPI.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today